
CORSO DI ARCHITETTURE DEGLI ELABORATORI A.A. 2025/2026
PROF.SSA VALENTINA CASOLA

ARCHITETTURE ARM

Dr. Alessandra Somma
alessandra.somma@unina.it

STORIA DI ARM



- Negli anni '80 la società inglese **Acorn Computers** crea insieme alla BBC (British Broadcasting Corporation) un nuovo calcolatore personale chiamato **BBC Micro**.
- A partire da questo accordo, Acorn ebbe le risorse per sviluppare la serie di **calcolatori** Acorn **Archimedes**, basati su un nuovo processore RISC e commercializzati tra il 1987 e il 1997, ma non ebbe grande successo.
- Nel 1985, Acorn progetta il primo processore RISC commerciale di questa serie era chiamato **ARM** (**A**corn **R**isc **M**achine) che ebbe un immediato successo.
- Acorn non produceva fisicamente i chip. La VLSI Technology, «silicon partner», aveva la licenza per fabbricare e commercializzare i chip basati sulle specifiche di Acorn.
 - Il primo processore ARM1 richiedeva circa 25000 e girava ad una frequenza di 6MHz, molto meno costoso con il modello poco precedente del Motorola 68000 che ne aveva 70000.
 - Seguono ARM2 (1985) e ARM3 (1989).

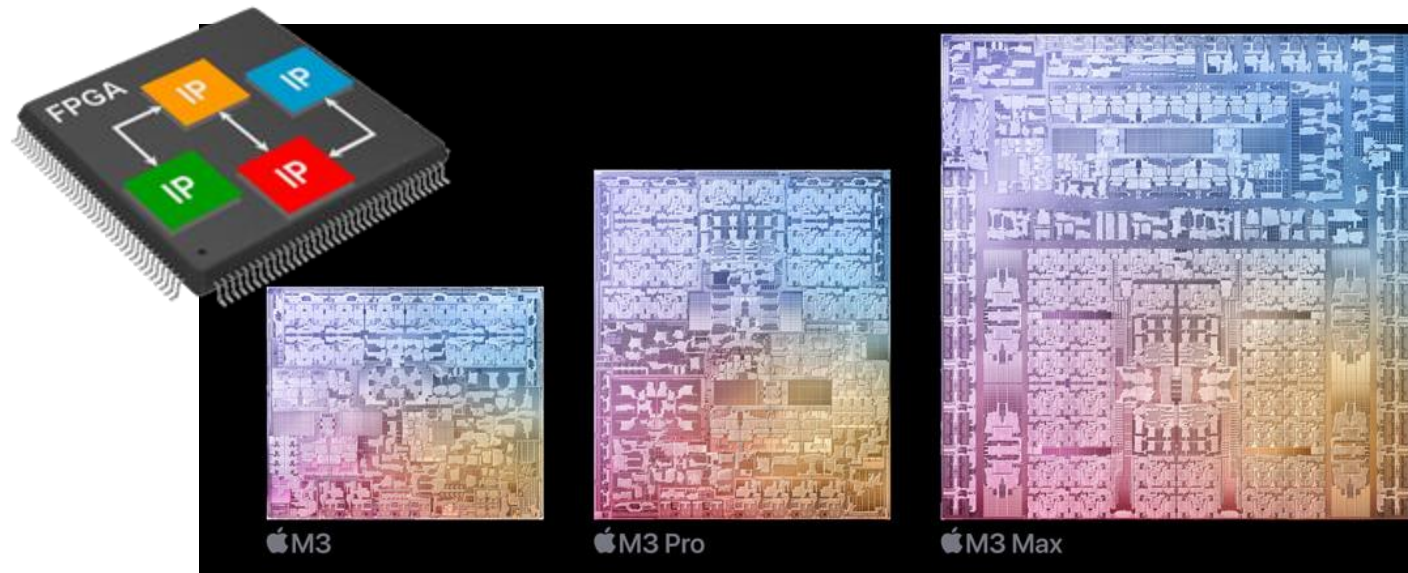


EVOLUZIONE DEI PROCESSORI ARM

- Verso la fine degli anni '80 - inizio '90, venne creata la società **Advanced Risc Machines Ltd**, a cui partecipavano Acorn, VLSI Technologies (adesso NVX Semiconductor N.V.) e Apple Computer (adesso Apple Inc.), poiché i processori ARM avevano ottenuto successo commerciale ma la sola Acorn non aveva abbastanza risorse per continuare.
- Il processore fu ribattezzato **Advanced Risc Machine** ed il primo prodotto fu **ARM6** (1992) che era una versione migliorata dell'ARM3.
- Da allora l'architettura ARM ha continuato ad evolversi con nuove funzionalità e migliori prestazioni
 - 2005: in quell'anno ARM vende 1,6 miliardi di licenze di chip (di cui 1 miliardo nei telefoni cellulari)
 - 2008: a gennaio raggiunge la soglia di 10 miliardi di chip
 - 2009: si stima che siano il 90% di tutte le CPU RISC a 32 bit
 - 2011: a gennaio raggiunge la soglia di 15 miliardi di chip
 - Oggi: ARM stima che i propri processori siano impiegati nel 95% degli smartphone, 80% delle fotocamere digitali e in generale nel 35% degli apparecchi elettronici.

ARM PROPRIETÀ INTELETTUALE

- Uno dei fattori essenziali del successo di ARM consiste nel meccanismo di licenza. La ARM Ltd. non produce il chip fisico, ma vende licenze per l'utilizzo dell'IP (**Intellectual Property**) relativo al processo del processore ARM.
- La società licenziataria può modificare il progetto per adattarlo ai propri scopi; produrre il chip microprocessore; integrare il microchip all'interno di un proprio sistema – SoC (**S**ystem **on** **C**hip) che integrano altri componenti come cache, GPU; vendere il progetto modificato ad altre società.



Ad esempio, a partire dal 2020, Apple Inc. realizza i SoC di serie M, architetture RISC basate su ARM e realizzate da Apple Silicon. Questo segna il passaggio da un'architettura Intel x86 ad una basata su IP ARM.

PROFILI ARM

- A partire dal 2005, ARM non vende più il suo nome come acronimo ma le tre lettere a-r-m hanno assunto tre significati che sono alla base della classificazione dei prodotti ARM. Il **profilo** è inteso come la **tipologia di sistemi per cui l'architettura è pensata**.
 - **A (Application)** profile) implementato nei Cortex-A idonei per applicazioni che necessitano di potenza di calcolo e flessibilità.
 - **R (Real-time)** profile) implementato nei Cortex-R, idonei per applicazioni di controllo che non richiedono unità avanzate presenti nel profilo A.
 - **M (Microcontroller)** profile) implementato nei Cortex-M per sistemi embedded semplici che prediligono la compattezza del codice binario.

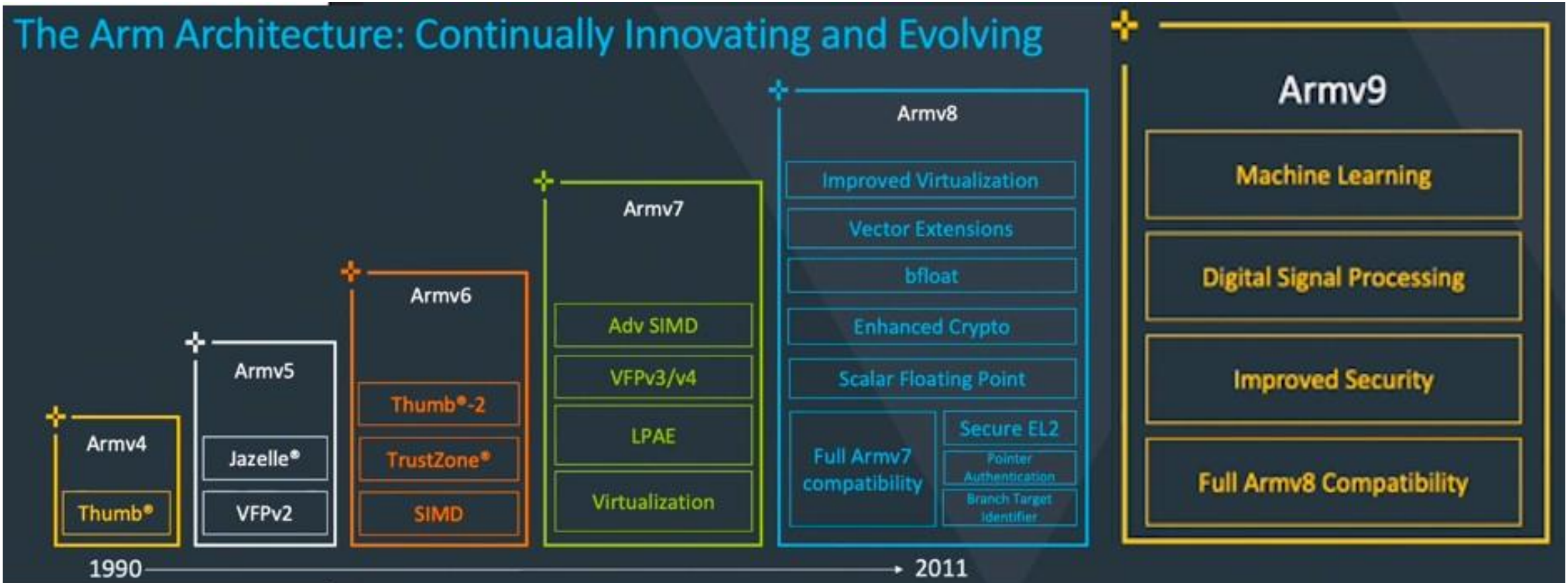


Le famiglie di processori ARM hanno essenzialmente lo stesso insieme di base di istruzioni macchina, tuttavia programmi compilati specificamente per un certo processore non funzionano necessariamente con un processore di un'altra famiglia.

VERSIONI ARM

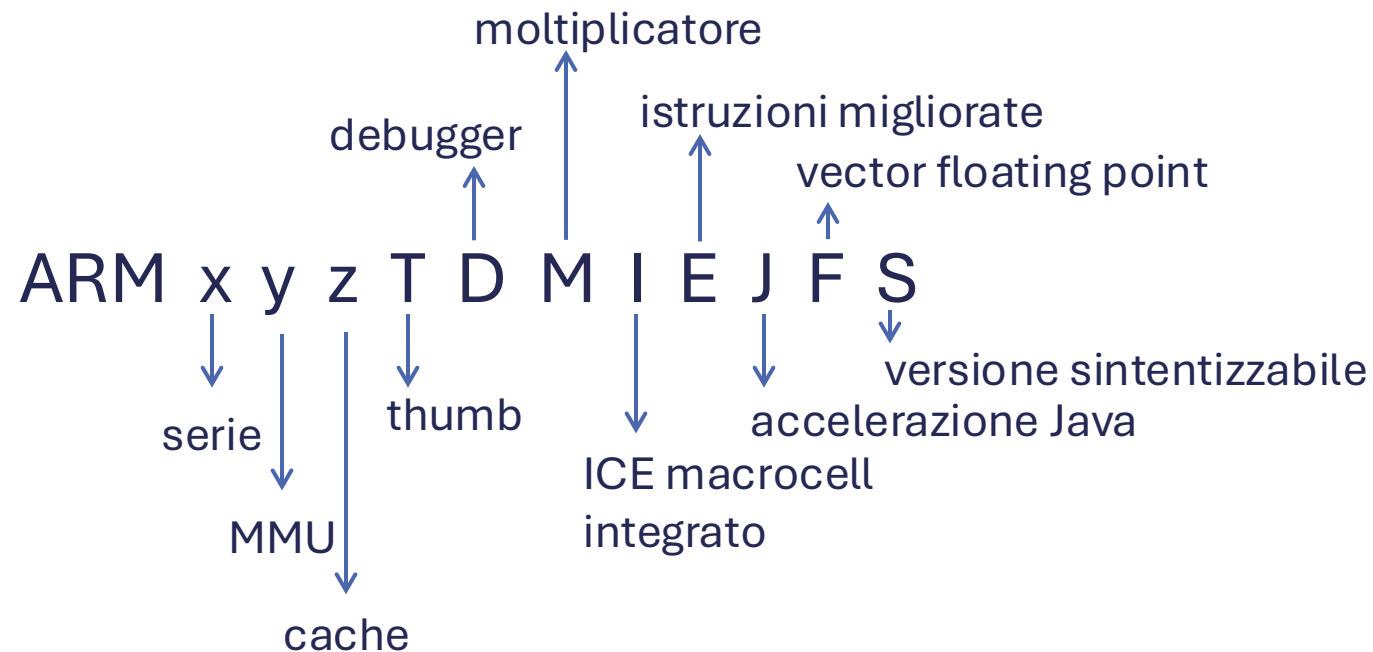
- Le versioni architetturali ARM sono i **modelli ISA** offerti nel corso degli anni.

Il **modello ISA** indica l'insieme di caratteristiche di un processore (**modello di programmazione, modi di indirizzamento e repertorio di istruzioni**). Queste informazioni sono rese disponibili al programmatore dai progettisti del processore in appositi manuali tecnici.



NOMENCLATURA DEI PROCESSORI ARM

- La sigla assegnata ai processori ARM ha un particolare significato:



LE ARCHITETTURE ARM: CARATTERISTICHE

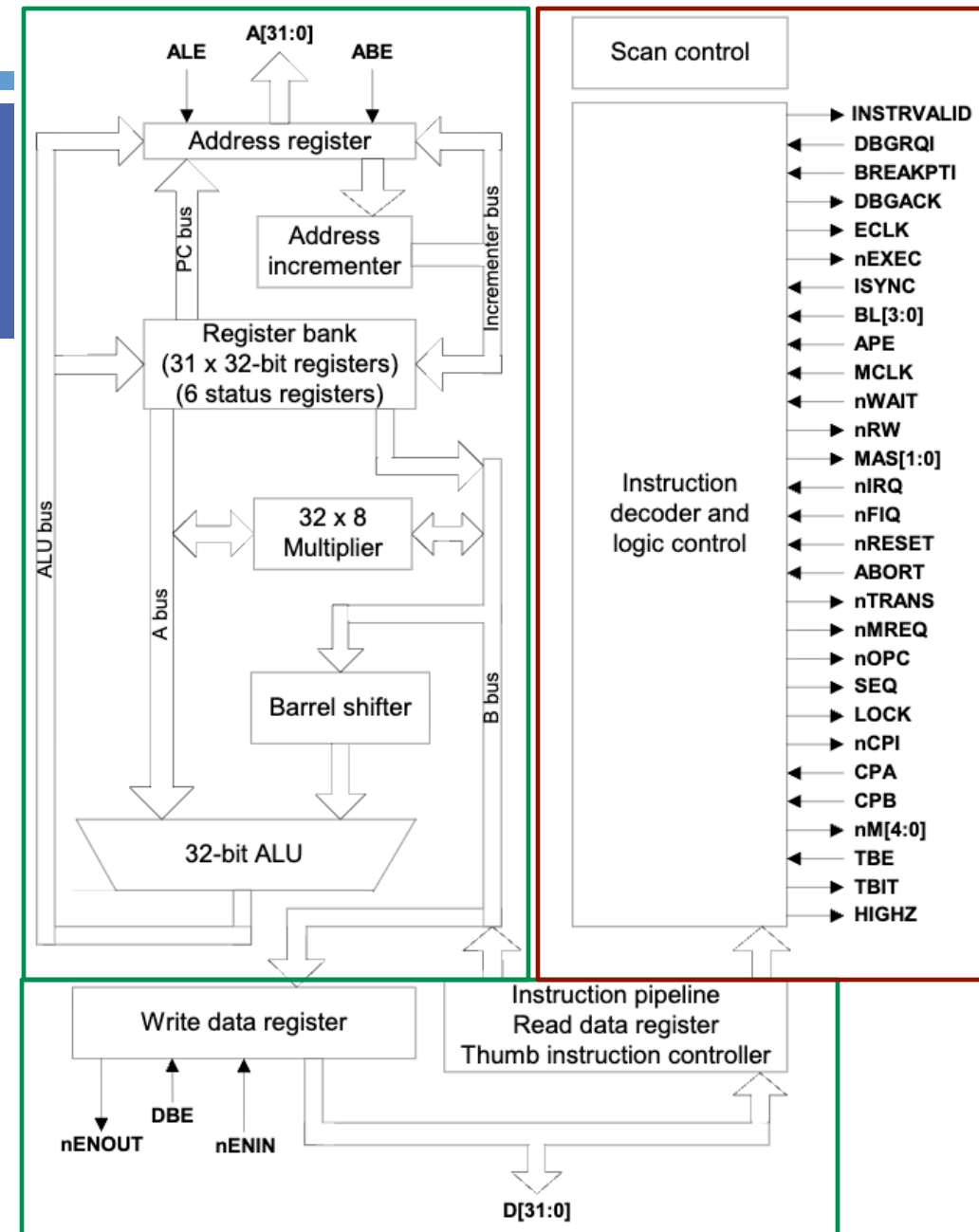
- L'architettura ARM è di tipo **RISC** (Reduced Instruction Set Computer) poiché integra questi aspetti tipici delle architetture RISC:
 - Un banco di registri di grandi dimensioni
 - **Architettura load/store** in cui le operazioni di data-processing sono eseguiti solo sul contenuto dei registri, non direttamente sul contenuto di locazioni di memoria
 - **Modi di indirizzamento semplici**, con indirizzi delle operazioni load/store determinati solamente dal contenuto dei registri e i campi dell'istruzione
 - **Istruzioni a lunghezza fissa** che semplificano l'attività di decodifica dell'istruzione (**32 bit per istruzioni ARM, 16 bit per istruzioni Thumb**)
- Inoltre, le architetture ARM offrono dei *miglioramenti rispetto alle architetture RISC* di base per avere un buon bilanciamento tra performance, dimensione del codice ridotta, riduzione dei consumi energetici e dell'area occupata su silicio:
 - controllo della ALU e dello shifter in ogni istruzione di data-processing per massimizzare l'utilizzo dell'ALU e dello shifter
 - **Modi di indirizzamento con auto-incremento e auto-decremento** per ottimizzare i cicli nei programmi
 - **Istruzioni multiple di load e store** per massimizzare il throughput di dati.
 - **Esecuzione condizionale** delle istruzioni per massimizzare il throughput di esecuzione

UN ESEMPIO: IL PROCESSORE ARM7DMI

- ARM ha ottenuto un enorme successo con la linea **ARM7** nel 1994 (con il processore ARM7TDMI), divenuto uno dei processori RISC più usati nei sistemi embedded per i successivi 15 anni:
 - La sigla TDMI sta per Thumb, JTAG Debug, fast Multiply, In-circuit debug.
 - Utilizza il set di istruzioni di ARMv4T che aveva introdotto le istruzioni **Thumb**, che è stato poi migliorato con le versioni ARMv5TE e ARMv6.
 - Nota:** bene che ARMv5TE è la versione avanzata di ARMv4T che introduce principalmente le istruzioni DSP, ARMv6 aggiunge il supporto alle SIMD e la gestione di istruzioni Thumb in maniera interlacciata. Le istruzioni per DSP e SIMD sono al di fuori degli obiettivi del corso.
 - Non tutti i manuali dei processori ARM sono disponibili online (poiché ARM vende IP!), infatti è possibile trovare il manuale della ARMv6-M (variante della v6 rivolta a profili embedded e supporta il solo set di istruzioni Thumb), ma non il manuale ARMv6.
 - In questo corso, le versioni che prendiamo come riferimento sono **ARMv4-ARMv6**. Data la mancanza di un manuale per ARMv6, prendiamo come riferimento «*ARM7TDMI Technical Reference Manual*». In *linea generale*, l'architettura che studieremo contiene a grandi linee quello che *tipicamente* contiene un core ARM. **Nota:** al variare di versioni e/o profili e a seconda di come il processore viene istanziato su silicio, l'architettura può variare! L'obiettivo del corso è conoscere l'architettura di massima di un processore ARM e la sua programmazione in assembly.
 - ARM offre un manuale di riferimento generale «*ARM Architecture Reference Manual*» consultabile online.

ARCHITETTURA DI MASSIMA DI UN PROCESSORE ARM

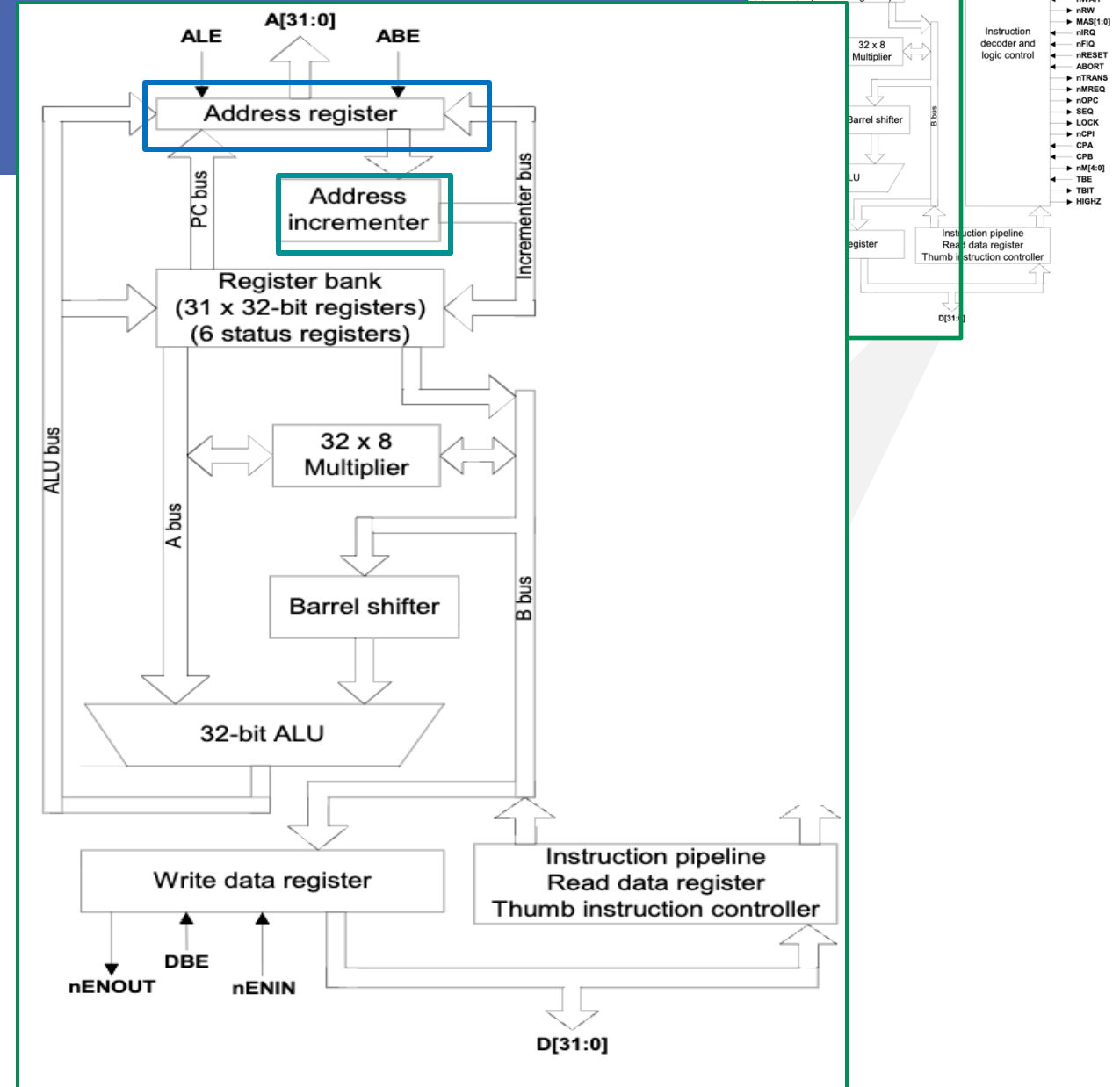
- L'architettura di un processore ARM è composta da una parte operativa ed una parte di controllo (in generale è vero per tutti i sistemi di elaborazione).
 - La **parte operativa** (anche detta **datapath**) è la parte che svolge tutte le operazioni aritmetiche, logiche e di trasferimento dati.
 - A sinistra nella figura c'è la parte dedicata alle operazioni **aritmetico-logiche** e di **indirizzamento**.
 - In basso a sinistra c'è la parte dedicata al **pipelining delle istruzioni** ed il registro dei dati letti dalla memoria.
 - La **parte di controllo** (UC) è la parte del sistema che è responsabile della gestione delle operazioni eseguite dall'unità operativa.
 - A destra della figura c'è il blocco di **Instruction Decoder and logic control** che genera tutti i segnali di controllo verso l'esterno ed eventualmente prende segnali di ingresso.
 - In alto a destra è presente un blocco di *scan control* usato in fase di test di produzione.



ARM:

PARTE OPERATIVA (1)

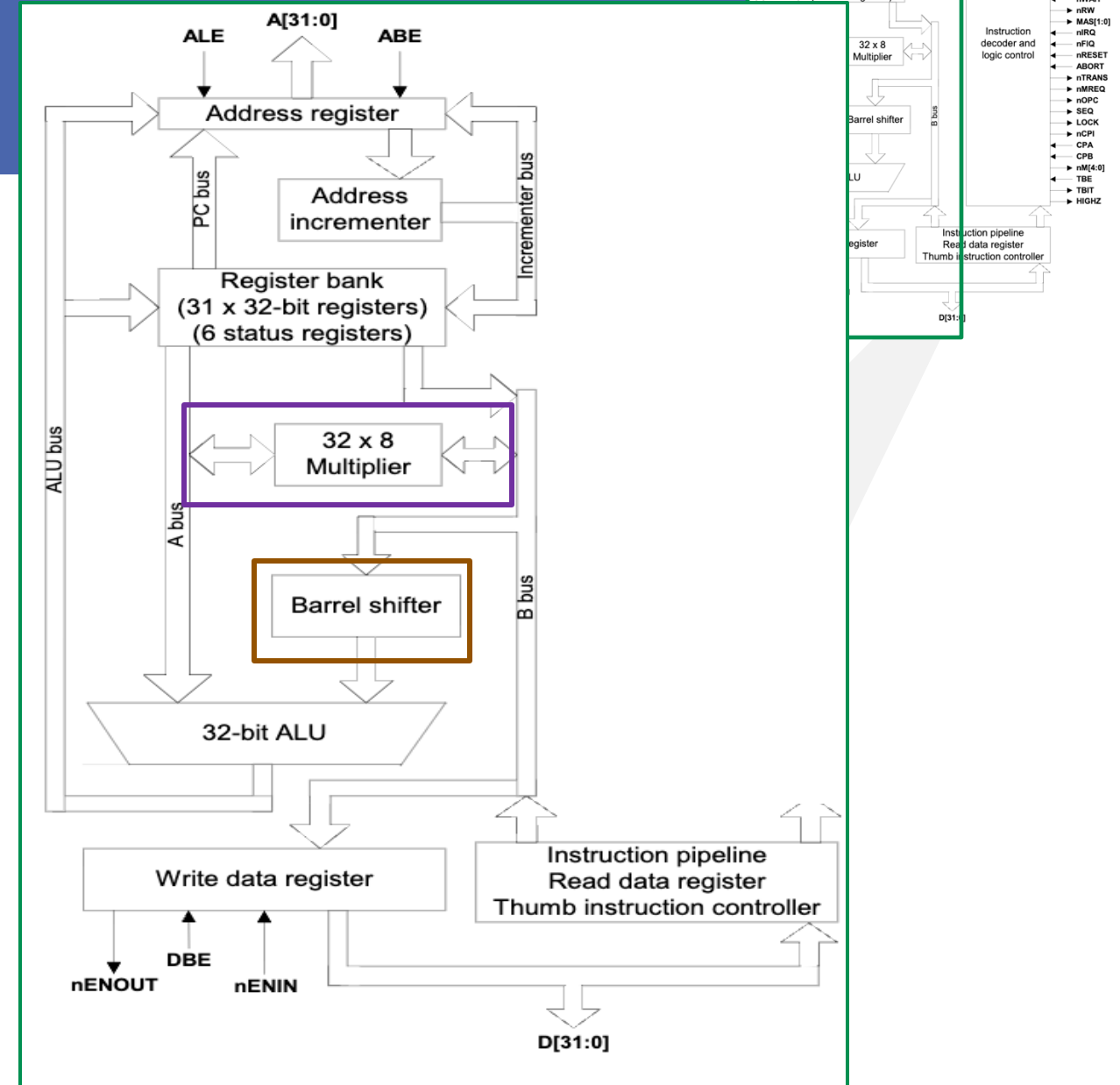
- **Address Register** è il registro che contiene l'indirizzo corrente, posizionato sul bus indirizzi esterno **A[31:0]**. E' un registro a 32 bit per accesso alla memoria: riceve un nuovo indirizzo dall'address incrementer tramite il «**PC bus**» e lo manifesta sul bus.
 - Segnale ALE (Address Latch Enable) abilita la messa in uscita dell'indirizzo.
 - Segnale ABE (Address Bus Enable) abilita o disabilita le linee di indirizzo.
- **Address Incrementer** è una piccola unità aritmetica dedicata che somma un valore fisso (tipicamente 4 o 2) all'indirizzo. Prende in ingresso il valore del PC e fornisce l'uscita all'address register tramite l'«**increment bus**».
- Serve per generare PC+2 o PC+4 a seconda della modalità Thumb o ARM.



ARM:

PARTE OPERATIVA (3)

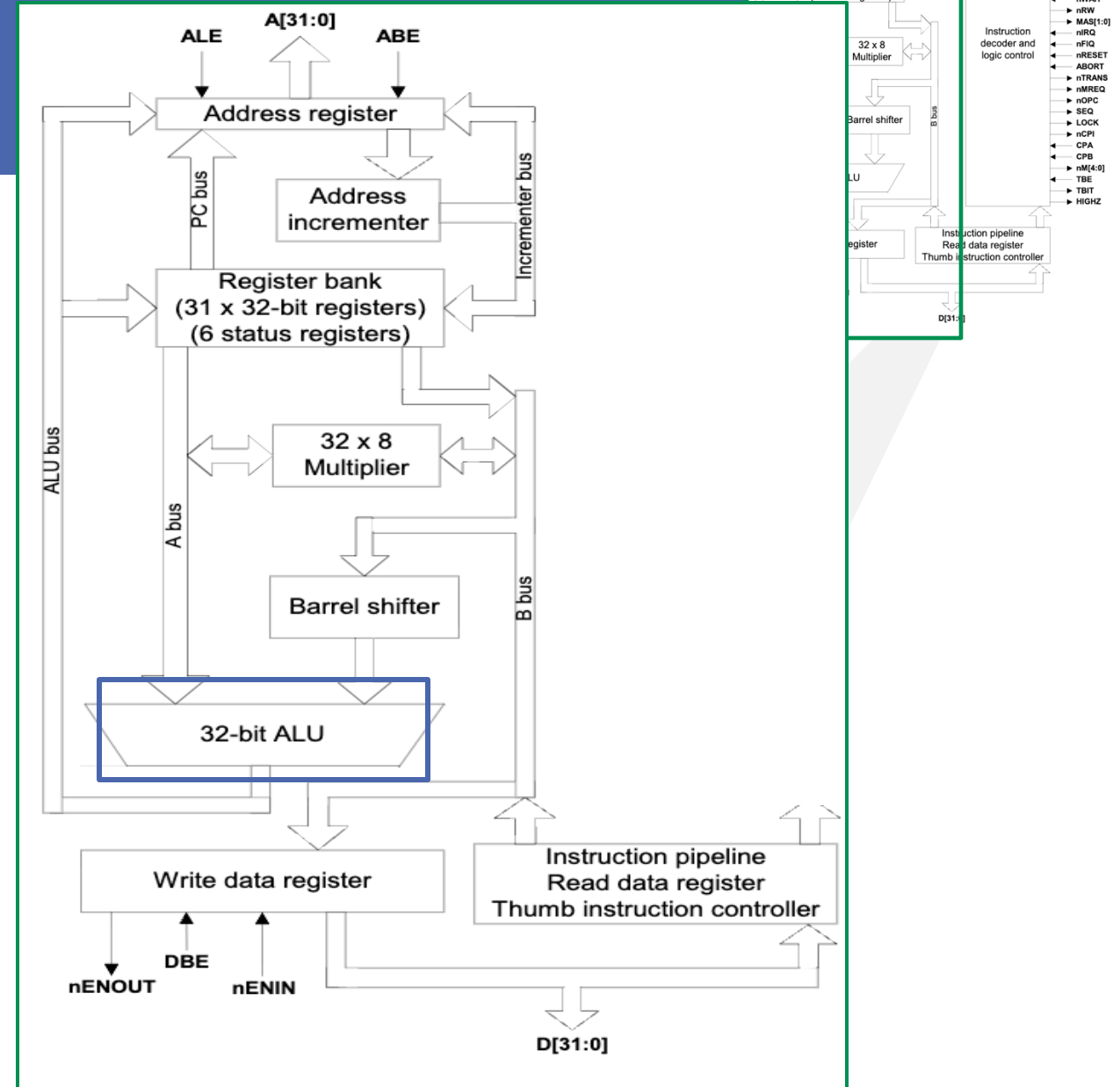
- **32x8 multiplier** è l'unità che esegue le moltiplicazioni tra un operando a 32 bit che viene dai bus A/B e un operando ad 8 bit (dal bus B o decodificato dall'UC). L'uscita viene riposizionata sul B bus per entrare in ingresso alla ALU. L'obiettivo è realizzare operazioni di moltiplicazione in uno o pochi cicli.
- **Barrel shifter** è l'unità che sposta o ruota un valore a 32 bit. Può eseguire uno shift logico a destra/sinistra, shift aritmetici e rotazioni. Prende l'ingresso dal bus B, un valore di shift (da UC, da un registro o un immediato) e fornisce l'uscita verso uno degli ingressi ALU.
- Consente di fare in modo che quasi tutte le istruzioni dell'ISA ARM possano lavorare su un operando già shiftato/ruotato nello stesso ciclo dell'ALU.



ARM:

PARTE OPERATIVA (4)

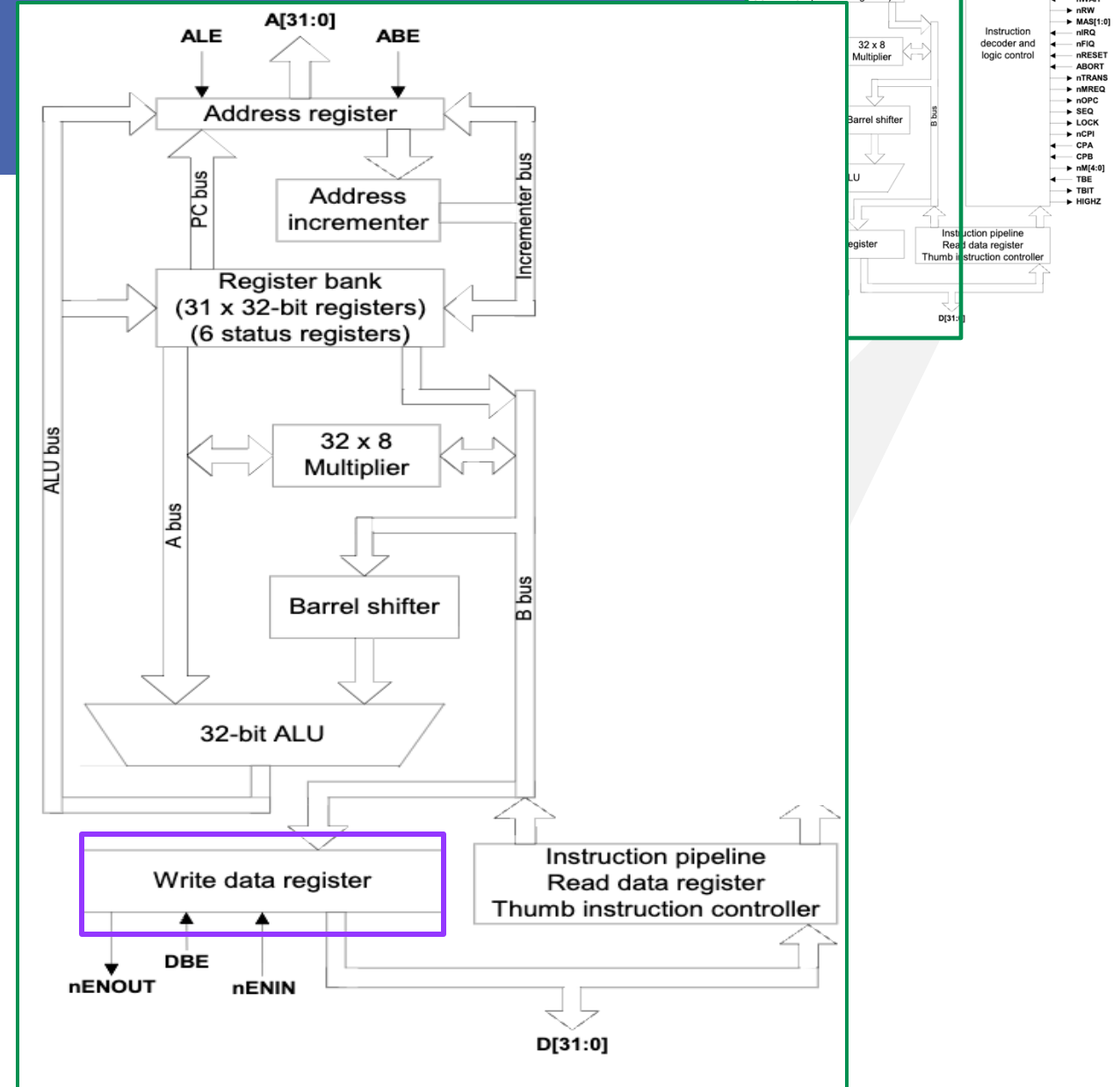
- **ALU** è l'unità aritmetico-logica che lavora a 32 bit. Prende come ingressi il primo operando dal bus A e il secondo operando che viene dal barrel shifter o dal moltiplicatore tramite il bus B.
- L'uscita dell'ALU viene posizionata sul bus interno che alimenta il register bank, il write data register (se il risultato deve andare in memoria).
- Esegue tutte le operazioni aritmetiche e logiche richieste dalle istruzioni ARM/Thumb.



ARM:

PARTE OPERATIVA (4)

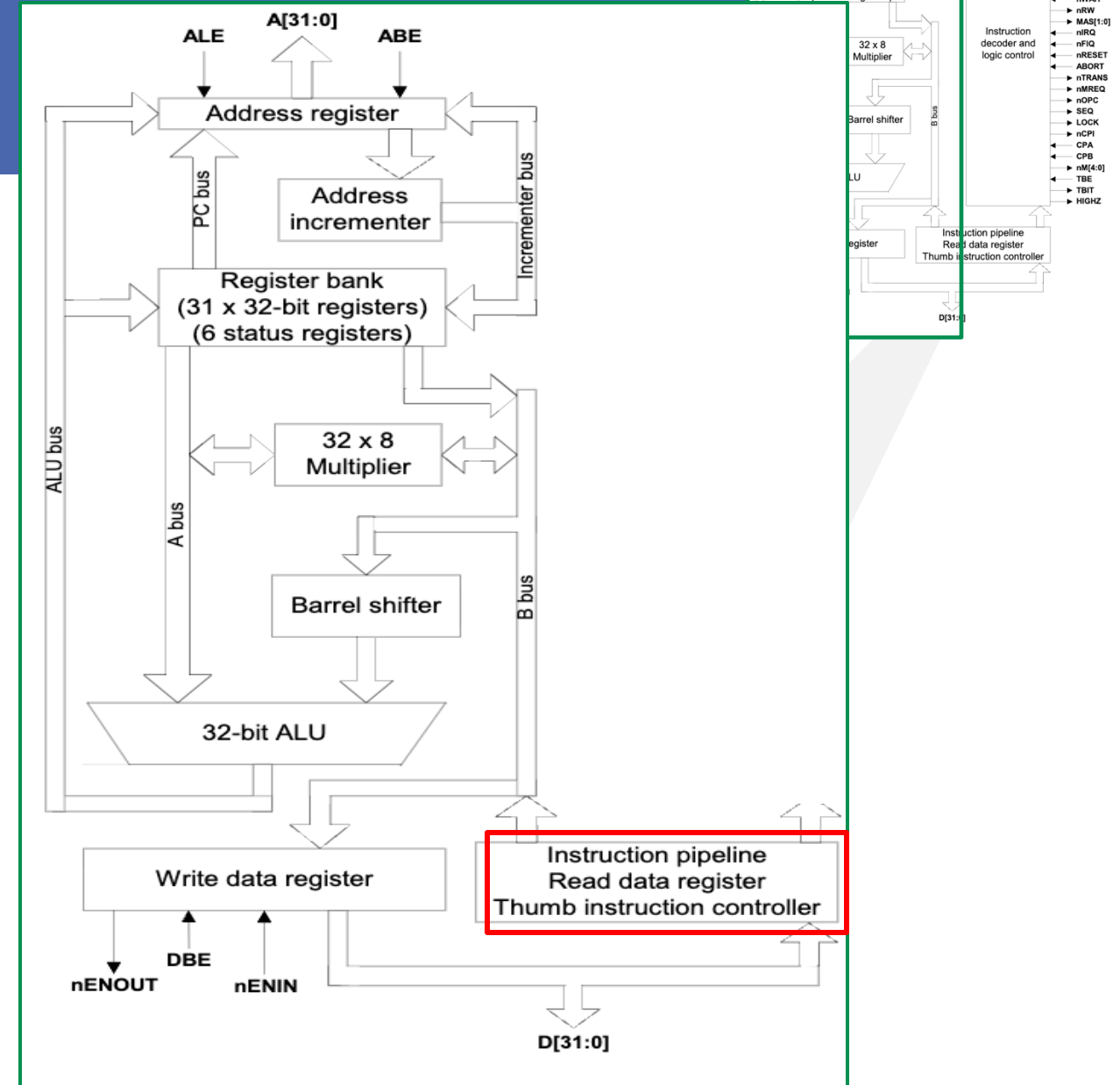
- **Write Data Register** è il **registro di appoggio dei dati da scrivere sulla memoria**: riceve il dato dall'ALU/o dai registri e lo mette sul bus D[31:0] quando i segnali di controllo lo abilitano.
 - nENOUT è il segnale che abilita l'uscita del dato sul bus. Se è 1 il dato del registro viene copiato sul bus, altrimenti se è 0 l'uscita viene messa in tristate (disabilitato).
 - nENIN è il segnale che abilita la cattura di un dato dal bus.
 - DBE (Data Bus Enable) è il controllo del buffer tristate.
 - Ha la funzione di disaccoppiare il core dal bus esterno, permettere il pipelining degli accessi in memoria e gestire eventuali attese (wait states).



ARM:

PARTE OPERATIVA (5)

- **Instruction Pipeline** prende le istruzioni dal bus dati D[31:0] (quando il trasferimento è di tipo «opcode») e le passa all'Instruction decoder a destra.
- **Read Data Register** è il registro nel quale viene salvato il dato letto dalla memoria. Da qui il dato può essere scritto nei registri generali tramite il datapath oppure avere un percorso dedicato a seconda dei segnali.
- **Thumb Instruction Controller** è la logica che gestisce le istruzioni Thumb (16 bit). Riceve parole da 16 bit (o due alla volta sul bus D[31:0]), le decompone/traduce in operazioni interne a 32 bit e coordina l'incremento del PC (+2/+4). Questa potrebbe anche essere considerata come *una piccola parte* della UC del processore.



INTRODUZIONE AL PIPELINING

- In generale, la velocità di esecuzione di un programma è influenzata da diversi fattori. Un modo per aumentare le prestazioni consiste nel progettare processore e memoria centrale in tecnologia microelettronica più veloce.
- L'alternativa consiste nel ristrutturare il processore in modo da renderlo maggiormente parallelo, cioè capace di svolgere più operazioni contemporaneamente.
- La metodologia del **pipelining** è molto efficiente per rendere il processore parallelo ed è un'idea che viene applicata anche nella nostra vita!
 - Ad esempio, nelle *catene di montaggio di un'industria automobilistica*, non si lavora un singolo veicolo alla volta, al contrario più veicoli sono disposti lungo la catena, ciascuno in uno stadio differente di lavorazione. In questo caso, il veicolo può impiegare un giorno o più per percorrere tutta la catena... MA la catena ogni pochi minuti può produrre un veicolo finito poiché ne contiene sempre numerosi in preparazione.
 - Se applichiamo lo stesso discorso al processore, possiamo pensare di **organizzare l'unità di controllo del processore in tre o più stadi**, in modo che **le diverse fasi di cui si articola l'esecuzione di un'istruzione possano operare in parallelo**.
 - In questo corso, facendo riferimento all'ARMv4-ARMv6, consideriamo un pipelining a 3 stadi. Le versioni più recenti possono avere fino a 15 stadi di pipelining.

ARMV4-V6: PIPELINE A 3 STADI (1)

- Il meccanismo di pipelining è quindi usato tipicamente dai processori RISC per velocizzare l'esecuzione di un'istruzione: mentre un'istruzione viene prelevata (fetch) altre istruzioni sono in fase di decodifica o di esecuzione. Consideriamo **una pipeline a tre stadi**:



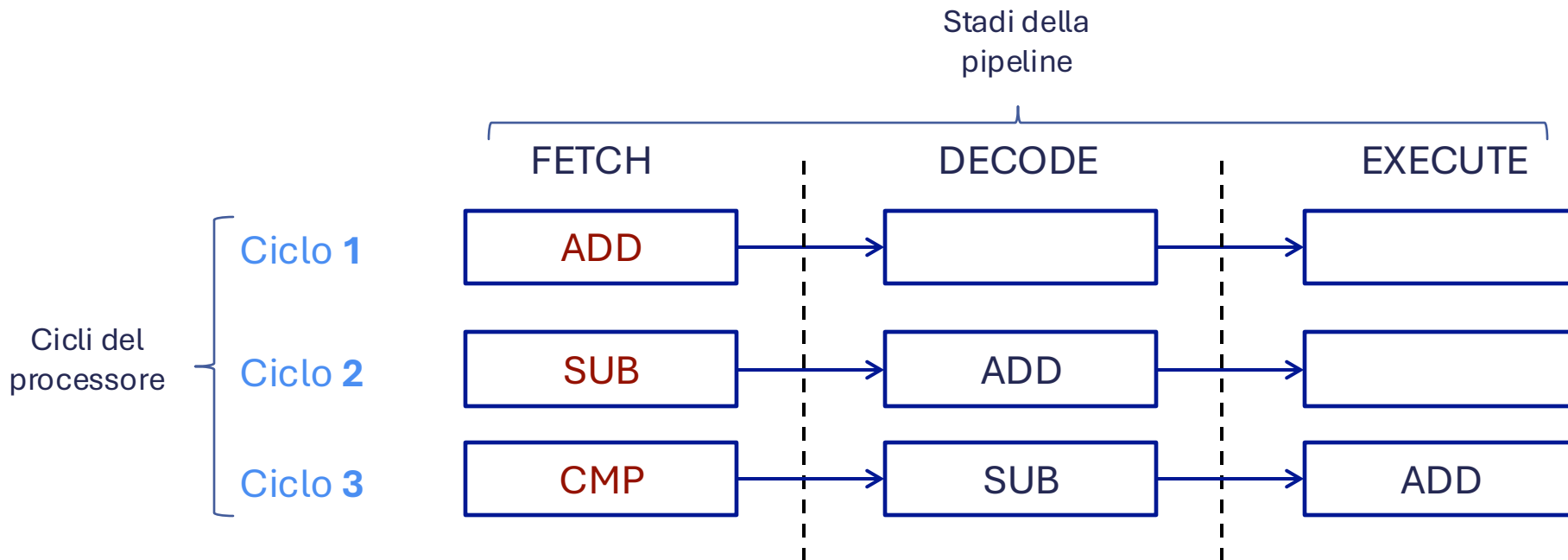
- Fetch* carica l'istruzione dalla memoria
- Decode* identifica l'istruzione da eseguire
- Execute* processa l'istruzione e scrive i risultati sui registri.
- Ogni istruzione richiederà comunque un ciclo intero (fetch - decode - execute) per terminare. Facciamo un esempio...

ARMV4-V6: PIPELINE A 3 STADI (2)

- Consideriamo le tre istruzioni assembly:

i_0 0x8000 ADD xxx
 i_1 0x8004 SUB xxx
 i_2 0x8008 CMP

Le tre istruzioni devono essere posizionate nella pipeline a 3 stadi in **maniera sequenziale**.



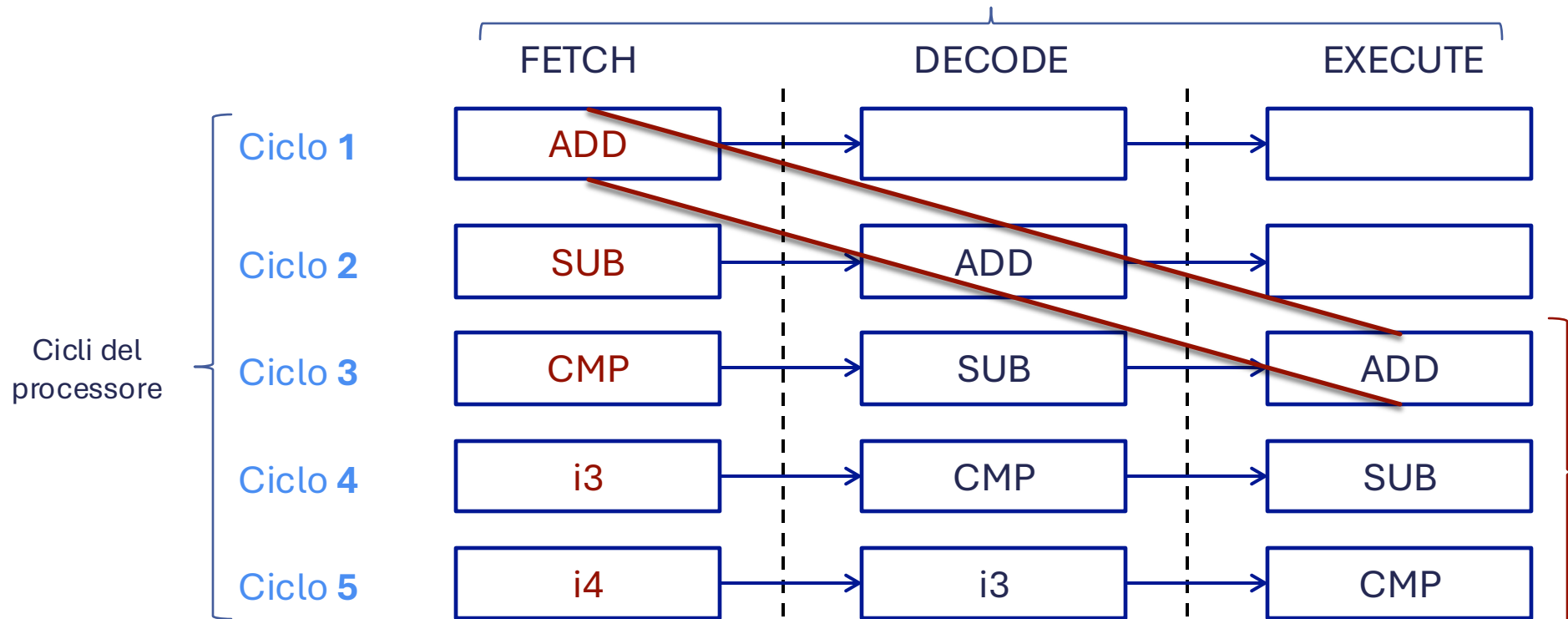
Questa operazione è chiamata **filling the pipeline**

ARMV4-V6: PIPELINE A 3 STADI (3)

- Consideriamo le tre istruzioni assembly:

i_0 0x8000 ADD xxx
 i_1 0x8004 SUB xxx
 i_2 0x8008 CMP

Le tre istruzioni devono essere posizionate nella pipeline a 3 stadi in **maniera sequenziale**.

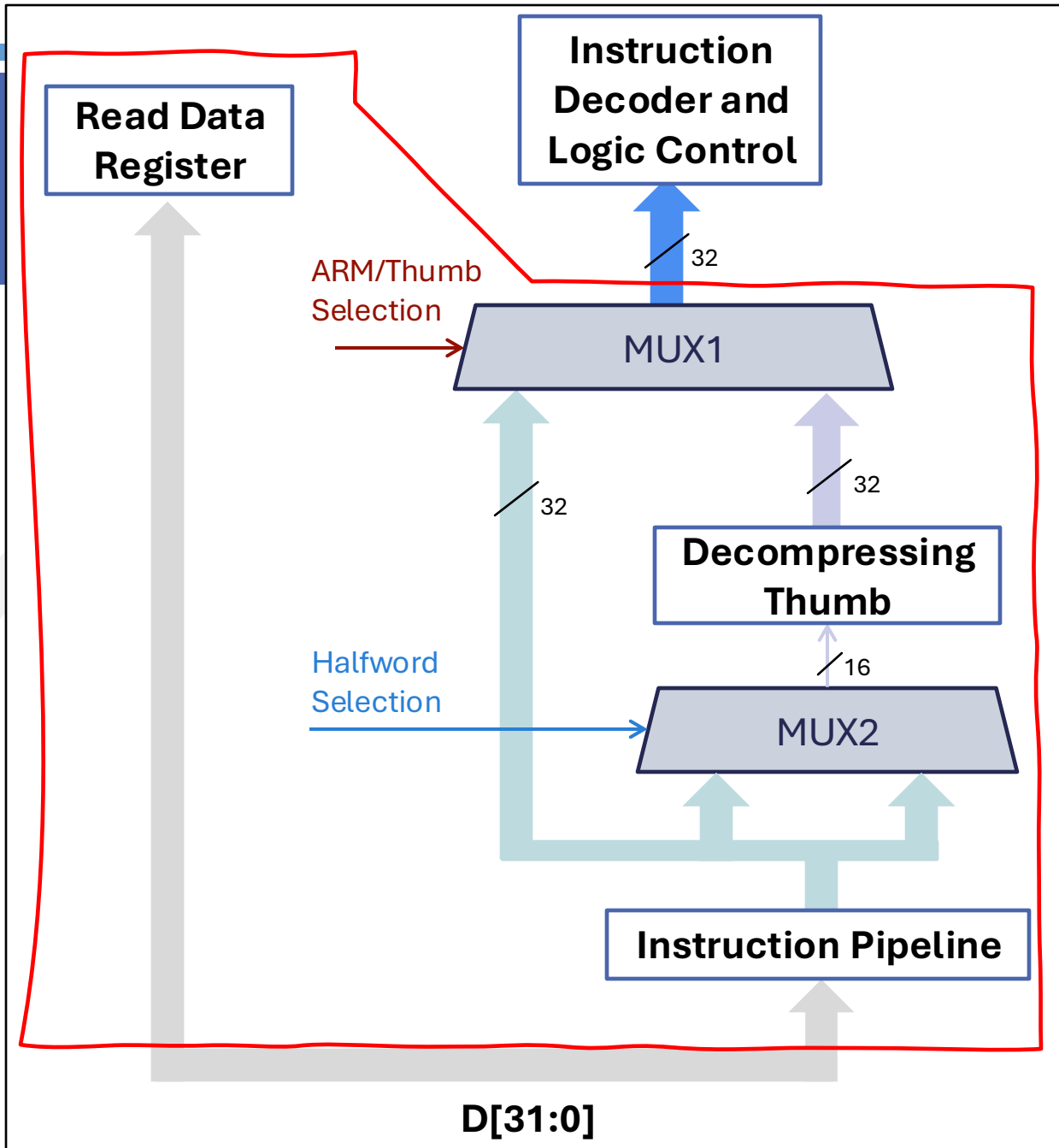
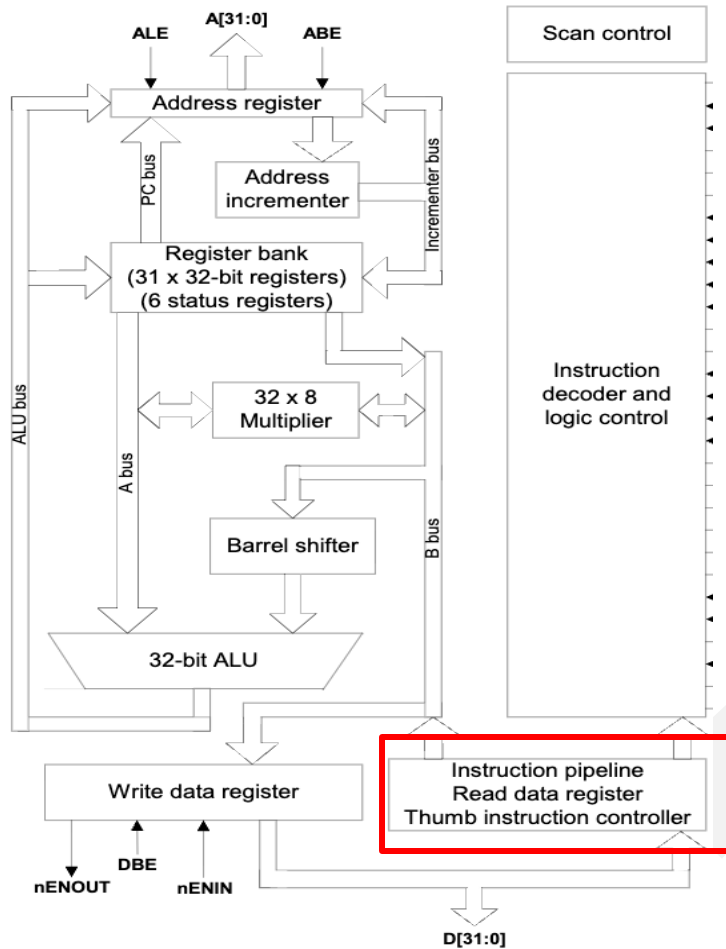


Ad ogni ciclo, il processore completa l'esecuzione di un'istruzione. Al ciclo 3, esegue la ADD, al ciclo 4 esegue la SUB, al ciclo 5 esegue la CMP e così via. Se ogni colpo richiede un tempo $T \rightarrow$ ogni T il processore esegue un'istruzione. L'istruzione viene eseguita passando attraverso tutti gli stadi della pipe (3 in questo caso).

PIPELINE A 3 STADI CONSIDERAZIONI

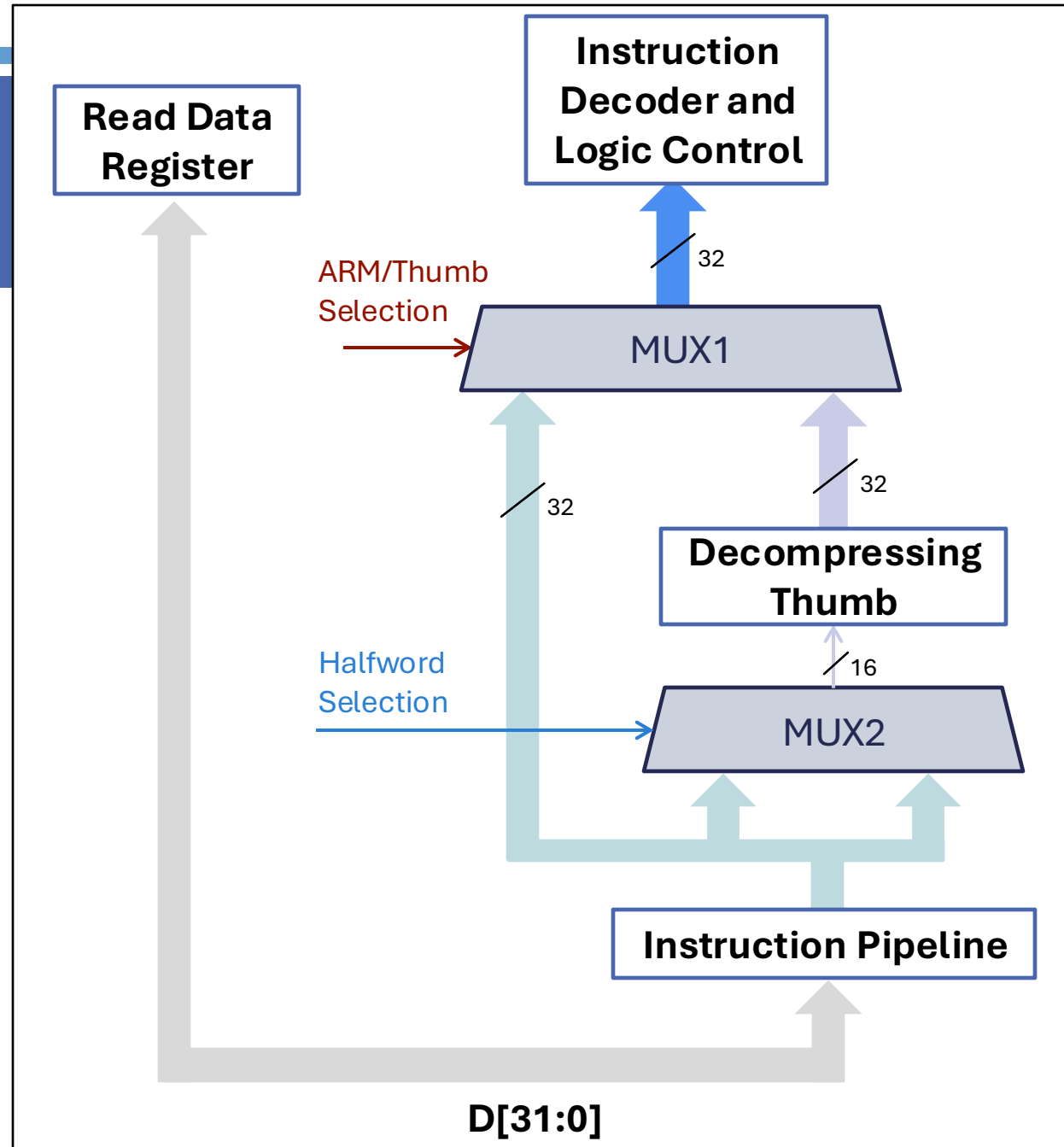
- Con il pipelining, **il processore è in grado di eseguire (idealmente) una istruzione ogni ciclo**. Si noti bene che **OGNI istruzione richiede sempre l'attraversamento (e quindi il passaggio) dei tre stadi**.
 - Questo vuol dire che se il ciclo del processore dura $1T$ e la pipeline è profonda $N=3$, il processore è in grado di eseguire una istruzione ogni $1T$, ed ogni istruzione attraversa $N=3$ stadi per poter essere eseguita.
- Al crescere della lunghezza della pipeline, è possibile ridurre la quantità di lavoro che il processore deve compiere ad ogni stadio, che consente al processore di arrivare a frequenze più elevate e quindi di incrementare le sue performance.
 - La latenza di ogni istruzione (cioè il tempo che passa dall'inizio del fetch dell'istruzione al termine della sua esecuzione) può essere pressoché la stessa, ma la capacità di lavoro svolta dal processore (throughput) è idealmente N volte superiore, dove N è la profondità della pipe.
- **Nota:** la crescita della pipeline aumenta non solo le performance, ma anche la possibilità che esistono delle dipendenze tra certi stadi della pipeline.
 - Immaginiamo che nel nostro esempio la SUB ($i1$) debba lavorare sul risultato dell'operazione di ADD ($i0$). Fintanto che la ADD non completa l'operazione, la SUB non può essere eseguita. Altri problemi sono creati dalle istruzioni di salto.
 - In questo corso, tralasciamo i meccanismi che servono per la gestione dei cosiddetti **hazard**, ma è importante sapere che è possibile da software scrivere codice che consenta di ridurre (alcune) dipendenze di tipo dato usando il cosiddetto instruction scheduling → il modo in cui scriviamo il codice impatta sulle performance della nostra macchina!

ARM: INSTRUCTION PIPELINE AND CONTROLLER



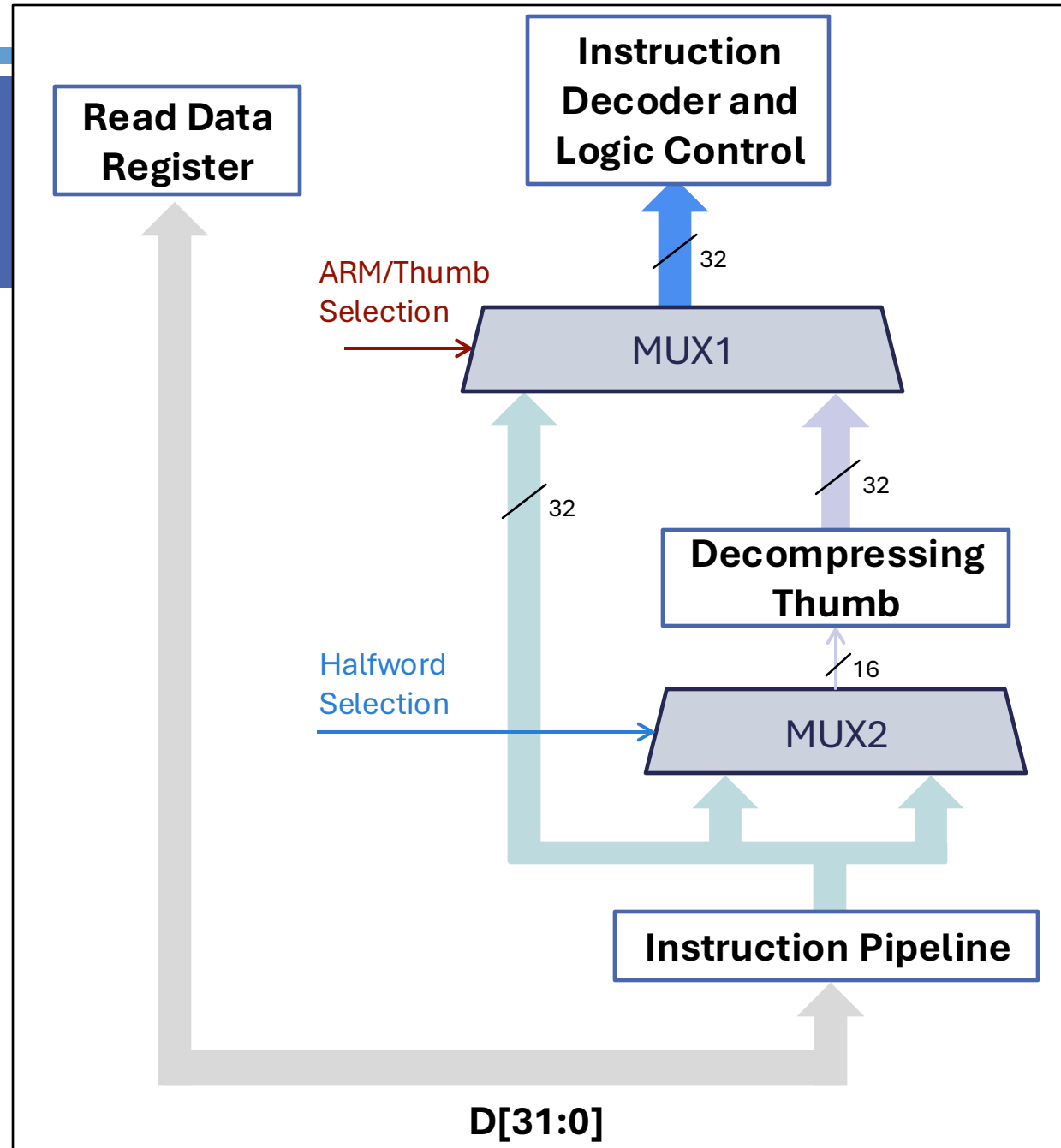
ARM: INSTRUCTION PIPELINE AND CONTROLLER

- Vediamo come l'istruzione dalla memoria arriva all'unità di controllo (Instruction Decoder and Logic Control).
- Quando il processore effettua un accesso in lettura alla memoria per prelevare un'istruzione, il valore presente sul bus dati D[31:0] viene immagazzinato nella pipeline delle istruzioni (**instruction pipeline**).
 - Se il dato che viaggia sul bus non è un'istruzione, il valore sarà indirizzato al **Read Data Register**, un registro buffer dedicato ai dati letti dalla memoria (che non sono istruzioni). Questo serve perché una load da memoria può richiedere più cicli di attesa e un tempo variabile.
 - Se si tratta di un'istruzione prosegue il percorso di decodifica.



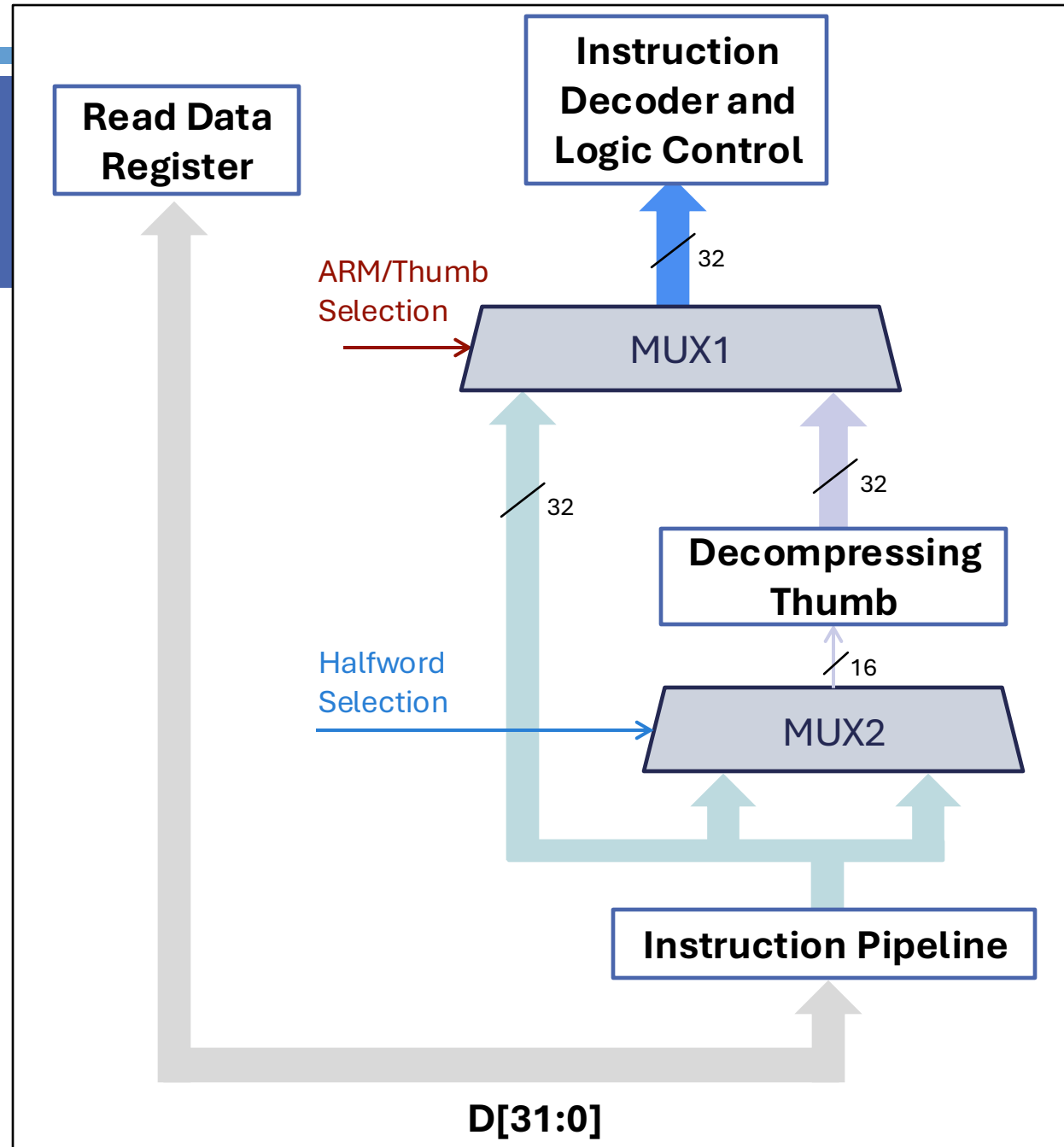
ARM: INSTRUCTION PIPELINE AND CONTROLLER

- Il **MUX1** decide se l'istruzione deve essere trattata come una normale istruzione **ARM** a 32 bit oppure deve essere inviata all'unità di decompressione **Thumb** che gestisce le istruzioni a 16 bit.
 - La selezione è effettuata tramite il **bit T** che si trova nel registro di stato del processore (CPSR): se $T=0$, la modalità è ARM, se $T=1$ la modalità è Thumb.
- Se il processore è in modalità Thumb ($T=1$), l'istruzione letta dalla memoria è lunga 16 bit, ma il bus dati fornisce sempre 32 bit alla volta!
 - Il **MUX2** seleziona l'**halfword meno significativa** $D[15:0]$ o l'**halfword più significativa** $D[31:16]$.
 - Questo MUX2 è pilotato tramite il segnale Halfword Selection (che deriva dal PC).
 - Questo serve ad assicurare che l'istruzione Thumb venga prelevata correttamente anche quando si trova nella parte alta del bus.



ARM: INSTRUCTION PIPELINE AND CONTROLLER

- Dopo la selezione della metà corretta della word, l'istruzione Thumb passa nel blocco **Decompressing** che riallinea correttamente la halfword selezionata e converte l'istruzione da 16 bit in un formato intero a 32 bit, equivalente all'istruzione ARM corrispondente.
- Con questa espansione, l'unità di controllo riceve sempre istruzione in formato uniforme a 32 bit indipendentemente dalla modalità di esecuzione!
- Se la modalità è Thumb, viene comunicato all'address incrementer che il PC deve avanzare di 2 byte e non di 4, come accade invece per le istruzioni ARM.



MODELLO DI PROGRAMMAZIONE: REGISTRI INTERNI

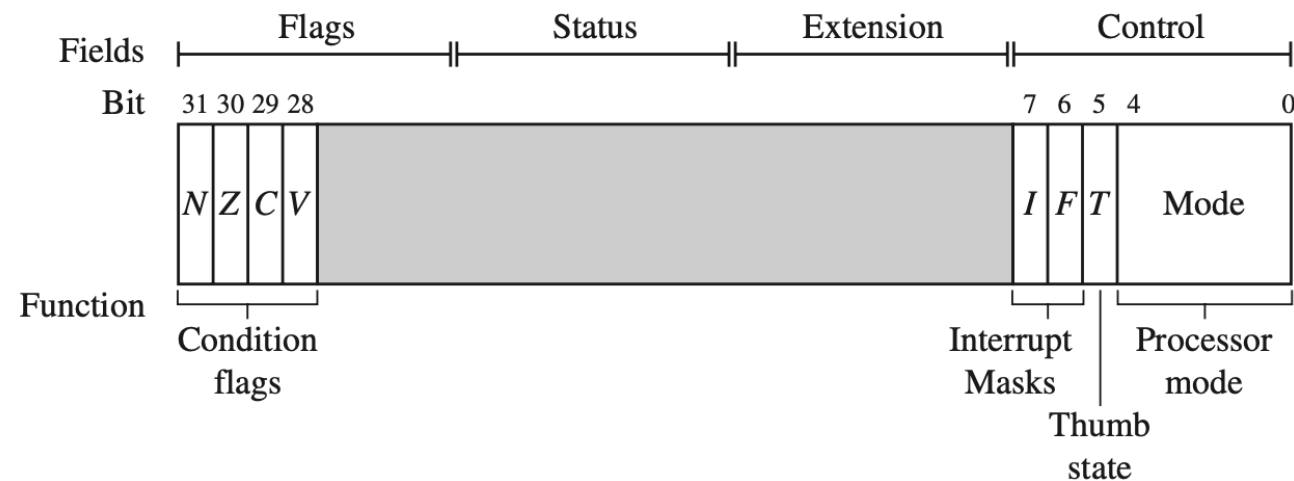
Si noti bene che stiamo parlando di registri VISIBILI al programmatore. Gli altri registri del banco visti nel datapath sono presenti ma accessibili nelle modalità di esecuzione privilegiata.



- Un processore conforme all'ISA ARMv6 possiede sedici (+1) **registri interni (R0-R15) da 32 bit utilizzabili da programma** (visibili al programmatore).
 - Il tempo di accesso ai dati mantenuti nei registri interni del processore è molto inferiore al tempo richiesto da un accesso in memoria (cache o centrale), pertanto l'utilizzo di tali registri è conveniente per mantenere dati temporanei dell'elaborazione.
 - I registri **R0-R12** sono **general purpose**, utilizzabili dal programmatore per memorizzare dati temporanei della elaborazione.
 - Il registro **R13** ha la funzione di **Stack Pointer**.
 - Il registro **R14** ha la funzione di **Link Register**, cioè viene usato all'entrata di un sottoprogramma per mantenere memoria dell'indirizzo di ritorno.
 - Il registro **R15** ha la funzione di **Program Counter**, cioè contiene l'indirizzo della word in memoria dalla quale sarà prelevato il codice della prossima istruzione. Dovendo ciascuna istruzione essere collocata ad un indirizzo multiplo di 4, i due bit meno significativi di R15 conterranno sempre il valore 0 (in caso modalità arm).
 - Anche il registro R14 può essere considerati general purpose, l'R13 solo se il processore è in modalità ARM, mentre *il registro R15 non è classificabile come general purpose*.
 - In aggiunta ai sedici registri R0-R15 esistono due registri di stato (**registro CPSR e SPSR**) che contengono lo stato corrente e salvato del programma rispettivamente.

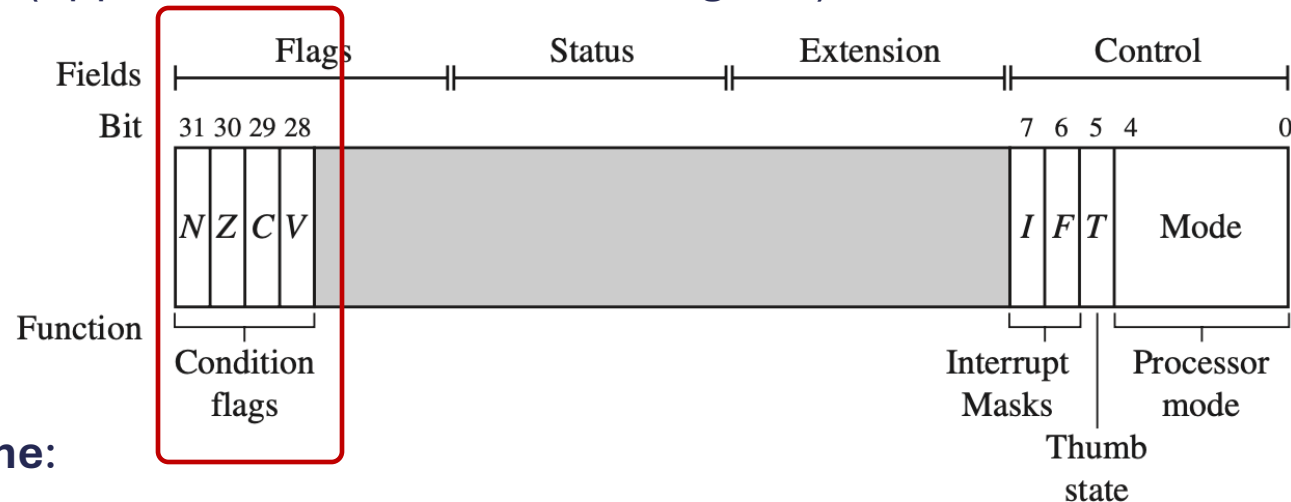
REGISTRO DI STATO CPSR

- Il registro **CSPR (Current Program Status Register)** è il **registro di stato**. Nelle versioni da ARMv7 in poi ha preso il nome di APSR (Application Processor Status Register). Noi utilizzeremo il nome CPSR.



REGISTRO DI STATO CPSR

- Il registro **CPSR (Current Program Status Register)** è il **registro di stato**. Nelle versioni da ARMv7 in poi ha preso il nome di APSR (Application Processor Status Register). Noi utilizzeremo il nome CPSR.

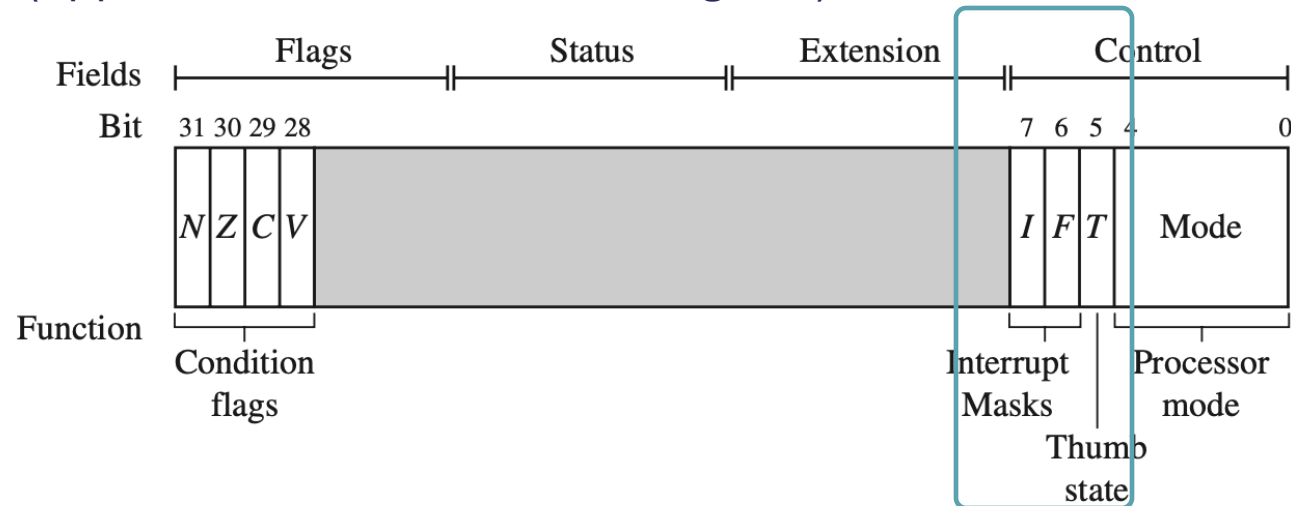


- Codici di condizione:**

- N (Negative)** se posto ad 1 indica che l'ultima istruzione abilitata a modificare il flag ha prodotto un risultato negativo (bit di segno 1);
- Z (Zero)** se posto ad 1 indica che l'ultima istruzione abilitata a modificare il flag ha prodotto un risultato nullo;
- C (Carry)** se posto ad 1 indica che l'ultima istruzione abilitata a modificare il flag ha generato un riporto;
- V (oVerflow)** se posto ad 1 indica che l'ultima istruzione abilitata a modificare il flag ha prodotto un overflow secondo le regole dell'aritmetica signed in complementi a due.

REGISTRO DI STATO CPSR

- Il registro **CPSR (Current Program Status Register)** è il **registro di stato**. Nelle versioni da ARMv7 in poi ha preso il nome di APSR (Application Processor Status Register). Noi utilizzeremo il nome CPSR.

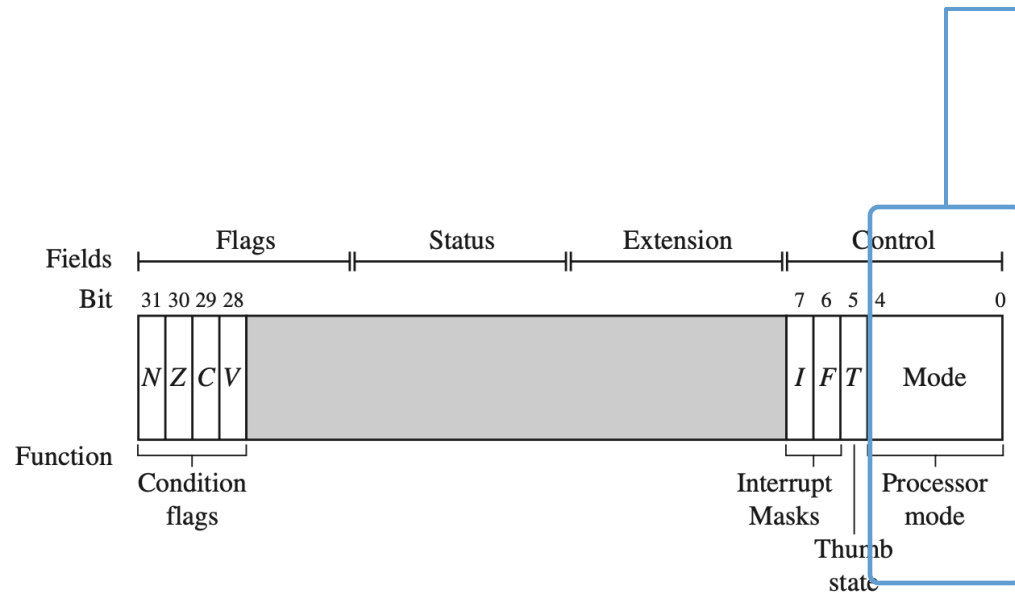


- I **flag I ed F** fungono da bit di abilitazione delle interruzioni (IRQ e FIQ) e non possono essere modificati quando il processore opera in modo utente.
- Il **flag T (Thumb)** se posto ad 1 indica che il processore opera conformemente al set di istruzioni Thumb a 16 bit.
- I cinque bit meno significativi del CPSR codificano il **modo operativo del processore**.

MODI OPERATIVI DEL PROCESSORE ARM

- Il **modo operativo** del processore determina le istruzioni eseguibili ed il banco di registri utilizzati da un processore ARM in un dato momento. I programmi utente sono eseguiti nel modo operativo «normale» (*user mode*), mentre istruzioni cosiddette privilegiate non possono essere eseguite in user mode. Dalla versione v4:
 - **User:** una modalità di esecuzione non privilegiata con accesso limitato alle risorse.
 - **System:** modo privilegiato per l'esecuzione del codice del sistema operativo.
 - **Supervisor:** modo privilegiato nella quale la CPU è portata in seguito ad un reset o ad un'istruzione di software interrupt (SWI)
 - **FIQ:** modo privilegiato in cui la CPU si porta quando viene servita una richiesta di interruzione ad elevata priorità (*fast interrupt*)
 - **IRQ:** modo privilegiato in cui la CPU si porta quando viene servita una richiesta di interruzione a priorità normale (*interrupt*)
 - **Abort:** modo privilegiato in cui la CPU è portata in seguito ad un accesso in memoria ad una locazione non consentita o inesistente.
 - **Undefined:** modo privilegiato in cui la CPU è portata in seguito al fetch di una istruzione non riconosciuta.
- Nelle implementazioni che includono le Security Extension esiste il modo Monitor, mentre nelle implementazioni che includono le Virtualization Extension esiste anche il modo Hypervisor.

MODI OPERATIVI DEL PROCESSORE ARM



MODE	CPSR bit[4:0]	MODE NAME
USR	10000	User Mode
FIQ	10001	FIQ Mode
IRQ	10010	IRQ Mode
SVC	10011	Supervisor Mode
ABT	10111	Abort Mode
UND	11011	Undefined Mode
SYS	11111	System Mode

BANCHI DI REGISTRI RISPETTO AI MODI DEL PROCESSORE

- Alcuni dei registri del processore sono **fisicamente duplicati in banchi distinti**, ciascuno dei quali è associato ad uno specifico modo operativo.
 - Questo approccio rende più veloci i cambi di contesto all'entrata e all'uscita delle Interrupt Service Routine (ISR).
 - Come visto nel datapath, un processore ARMv4-v6 presente in **totale 37 registri interni**, ma in un dato momento, **solo 16 di essi sono visibili al programmatore**.
- In system/user mode condividono lo stesso banco di registri.
- All'entrata in uno dei modi privilegiati, il contenuto del CPSR viene copiato in un altro registro detto **SPSR (Saved Program Status Register)**, specifico del modo privilegiato, per poterne eventualmente ripristinare il valore precedente.

User and system

<i>r0</i>
<i>r1</i>
<i>r2</i>
<i>r3</i>
<i>r4</i>
<i>r5</i>
<i>r6</i>
<i>r7</i>
<i>r8</i>
<i>r9</i>
<i>r10</i>
<i>r11</i>
<i>r12</i>
<i>r13 sp</i>
<i>r14 lr</i>
<i>r15 pc</i>

Fast interrupt request

<i>r8_fiq</i>
<i>r9_fiq</i>
<i>r10_fiq</i>
<i>r11_fiq</i>
<i>r12_fiq</i>
<i>r13_fiq</i>
<i>r14_fiq</i>

Interrupt request

<i>r13_irq</i>
<i>r14_irq</i>

Supervisor

<i>r13_svc</i>
<i>r14_svc</i>

Undefined

<i>r13_undef</i>
<i>r14_undef</i>

Abort

<i>r13_abt</i>
<i>r14_abt</i>

<i>cpsr</i>
-

<i>spsr_fiq</i>

<i>spsr_irq</i>

<i>spsr_svc</i>

<i>spsr_undef</i>

<i>spsr_abt</i>

STATO E SET DI ISTRUZIONI

- L'insieme di tutti i codici che un processore riconosce come **istruzioni si chiama set o repertorio di istruzioni del processore**. Lo stato del processore determina qual è il set di istruzione in esecuzione, dato che l'architettura supporta due instruction sets:
 - ARM è il set di istruzioni attivo solo se il processore è in ARM state.
 - Thumb è il set di istruzioni attivo solo se il processore è in Thumb state.
- L'ISA ARM consiste di un repertorio di istruzioni che possono essere classificate come segue:
 - Istruzioni di elaborazione dati;
 - Istruzioni di trasferimento dati (load/store)
 - Istruzioni di trasferimento dello status register;
 - Istruzioni di salto;
 - Istruzioni aritmetiche in virgola mobile e per i coprocessori;
 - Istruzioni per la generazione di eccezioni

STATO E SET DI ISTRUZIONI

- Nella modalità ARM, in memoria le istruzioni sono rappresentate mediante un codice a lunghezza fissa da 32 bit. Una singola istruzione occupa in memoria una word (**parola-codice**).
 - Le istruzioni sono collocate ad indirizzi allineati sul limite di parola, ovvero in word il cui primo byte ha come indirizzo un multiplo di 4.
- Nella modalità Thumb, le istruzioni sono rappresentate in memoria mediante codice a 16 bit, pertanto un codice Thumb occupa meno memoria di un codice ARM a 32 bit. Il passaggio da stato ARM a stato Thumb si effettua con le istruzioni di salto BX e BLX.
 - In stato Thumb, le istruzioni, pur operando sull'intero contenuto dei registri del processore, possono utilizzare solo i registri da R0 a R7.
 - Le istruzioni Thumb non possono essere soggette ad una condizione, ad eccezione dei salti condizionati.

RIFERIMENTI UTILI

1. ARM Architectures: <https://developer.arm.com/documentation/dui0471/m/key-features-of-arm-architecture-versions/about-the-arm-architectures>
2. ARM Architecture Reference Manual: <https://student.cs.uwaterloo.ca/~cs452/docs/ts7200/arm-architecture.pdf>
3. ARM7TDMI Technical Reference Manual